MoveMate Host Edition

Frontend Developer Guide

Build the phone-first app for hosts, guests and cleaners.

Version	2.0 (API contract v2) - October 2026
Read this if	You build everything in the `frontend/` folder.
Start here	Run `docker compose up --build`, open http://localhost:3000, press Load demo data.
Order of reading	Part A and B (shared) -> Part C (video matching) -> Part D (your own guide).

The one rule: the API contract in Part B is the promise between you and your teammate. Part A, B and C are identical in both guides on purpose. If you think the contract is wrong, change it together, in one pull request, never alone.

Part A. The shared picture (both guides contain this part, word for word)

A1. The product in one page

MoveMate Host Edition helps a short-term rental host in four ways. The host scans the home once and records its details, rooms, places and items. Guests get help from a photo. After every stay, a person walks the checklist and the system tells the host whether the home is ready for the next guest. An optional AI video check gives that person hints.

Workflow	Who	Input	Output
A. Baseline scan	Host	Photos of important items, short room-sweep videos	A confirmed inventory: items with name, quantity, reference photo, notes
B. Guest guide	Guest	One photo of an appliance, from a link, no login	Short step-by-step instructions for that exact appliance
C. AI video check (audit)	Cleaner or host	One video per room	Suggestions only: missing, moved, wrong count, could not verify, each with photo evidence
D. Inspection and readiness	Cleaner or host	A checklist per room, photos, counts, notes	Results, cases for anything off, and a readiness report: ready, ready with exceptions, or not ready

Two different checks, on purpose. The AI video check (C) only suggests. The inspection (D) is the human record and the only thing that creates results, cases and readiness. The link between them is one button that copies the AI's suggestions into the checklist as hints.

A2. Words we all use the same way

Word	Meaning
Property, Room, Place	A rental home, the rooms in it, and places inside a room at any depth (Living room > Glass cabinet > Top shelf). A place is stored as a room with a parent.
Item	One kind of thing the host cares about (a Nespresso machine, 4 wine glasses). Has a label, a quantity, a status, a type (kind), a condition and an availability.
Kind	fixed (TV, built in), movable (a remote), shared (bottle opener used in several rooms), consumable (toilet paper: counted, never 'missing').
Condition	new, good, worn, damaged, missing, under_repair. Every change is saved in a history with who, when, note and photo.
Availability	guest or staff_only. Staff-only items (cleaning spray) never appear in the guest guide.
Alternate place	Another place where an item may legitimately be (the TV remote on the TV console). Seeing it there is not 'misplaced'.
High value	A flag on an item. A photo is required before it can be marked OK in an inspection.
Reference crop	The cropped photo of an item taken at scan time. This is what the audit looks for later.
Suggested / Confirmed	The model proposes items (suggested). The host confirms them. Only confirmed items are used by guests and audits.
Baseline	The confirmed inventory made during the scan.
Audit (AI video check)	One AI check of a property from videos. Has findings. Suggestions only.

Word	Meaning
Inspection	One human checklist for a property: after checkout, after cleaning, or before check-in. Has a result per item and cleaning checks per room.
Cleaning status, inventory status	Two separate statuses per room in an inspection. A room can be clean but have a missing item, or complete but dirty.
Case (discrepancy)	A factual record opened when an inspection result is missing, misplaced or damaged. Open, investigating, resolved, with an append-only trail. Not a blame list.
Outcome	How a case ended: found, restored_to_place, repaired, replaced, recorded_missing.
Essential item (blocks readiness)	A case the host marks as blocking. The property cannot be ready while it is open.
Readiness	ready, ready_with_exceptions or not_ready. An inspection being complete does not mean the property is ready.
Exception	The host completes an inspection despite open checks, with a written reason. It never hides an essential gap.
Finding	One result of an audit: missing, misplaced, count_mismatch, unverified (and later extra). It has a resolution: open, found, confirmed or dismissed.
Resolution	open = nobody has looked. found = the cleaner says it turned up. confirmed = the host confirms a real loss. dismissed = the host says it is a false alarm.
Role	cleaner or host. In v1 there is no login, so the link carries the role (see A6).
Unverified / Not seen	Unverified: the video did not show the usual spot. Not seen: the person could not confirm it during an inspection. Neither is 'missing'. Only a person records a loss.
Mock mode	Fake but consistent model answers so the app works with no API keys.

A3. Architecture

```
phone (browser)  --->  Next.js frontend  (http://localhost:3000)
       |                        |
       |  fetch() with NEXT_PUBLIC_API_URL
       v                        v
  FastAPI backend (http://localhost:8000)  --->  Postgres (docker service "db")
       |   |
       |   +--> files in ./data  (served at /media/...)
       +------> model client: mock.py now, meta.py for real models
                (segmentation, labeling, reasoning, audit judging)
```

The browser never calls a model and never holds an API key. All model calls happen in the backend, behind one interface (`backend/app/services/models/base.py`). That is why the frontend can be built completely against mock mode.

A4. How we split the work so nobody blocks anybody

Area	Owner	Other person may...
`frontend/` (everything)	Frontend dev	read only
`backend/` (everything)	Backend dev	read only
`backend/app/schemas.py`, `frontend/lib/types.ts`, `docs/API_CONTRACT.md`	SHARED	change only together, in one PR, both approve

Area	Owner	Other person may...
`docker-compose.yml`, `.env.example`, `.gitignore`	Backend dev	ask first (a broken compose file blocks both of you)
`README.md`, `docs/`	Either	keep short and correct

- **The contract is the promise.** Part B lists every endpoint. If something is not in the contract, it does not exist yet. Ask for it.
- **Never rename or remove a field.** Add new optional fields instead. Both sides must keep working at every step.
- **Branches:** `feat/fe-...` for frontend work, `feat/be-...` for backend work, `contract/...` for shared changes. Merge to `main` at least daily.
- `main` **always runs.** `docker compose up --build` must work on `main` at all times. Test it before you merge.
- **10-minute sync every day.** Three questions: what I shipped, what I need from you, what I am changing in the contract.
- **Blocked? Use mock mode.** The backend already returns believable data (see B4), so the frontend never waits for real models.
- **Found a bug on the other side?** Open an issue with the request you sent, the response you got, and the expected response. Do not patch their folder.

A5. What we build in 10 days, and what comes later

The full product vision is large. For the hackathon we build a **slice that works end to end**, and we write down honestly what is left. Everything marked 'Built' is in the code and contract today. Everything marked 'Later' is not, and the UI must not pretend otherwise.

Area from the product spec	Hackathon slice	Status
Property details, house rules, procedures	Address, type, capacity, details, rules (free text), shown to guests	Built
Rooms and nested places	Rooms with a type; places at any depth; reference photo per room	Built
Items: placement, type, alternates, availability	Kind, usual place, alternate places, guest or staff-only, high value, minimum quantity	Built
Condition and history	Six conditions, each change saved with who, when, note and photo	Built
Visual inventory and printable sheet	Room reference photo, item crops, printable room sheet with tick boxes	Built
Inspections with required checks	Three kinds, per-room checklist, high-value photo, damaged needs note and photo, cleaning checks	Built
Exception approval	Host only, needs a note, never hides essential gaps	Built
Discrepancy cases	Detect, assign (host), search log, evidence, resolve with outcome, append-only trail	Built
Cleaning vs inventory status	Separate status per room	Built

Area from the product spec	Hackathon slice	Status
Readiness report	Ready, ready with exceptions, not ready; printable	Built
Consumables	Counted by quantity, low-stock flag in the report	Slice only. Stock ledger, restock lists and purchasing: Later
AI boundaries	AI only suggests: hints on checklist items, findings need a person	Built (rule enforced in code)
Accounts, roles, permissions	Two roles chosen by link (not secure)	Later: real login, per-property access, cleaner links scoped to one inspection
Bookings and calendar	Reservation reference typed by hand	Later
Notifications	None	Later
Maintenance jobs and costs	A damaged item just opens a case	Later
Dashboards, reports, exports	One readiness report	Later
Offline use, integrations	None	Later

Pitch line: 'We built the loop that matters: know what should be there, check it, record what is wrong, and say clearly whether the home is ready. Everything else is a layer on top.'

A6. Who can confirm what: the cleaner and the host

The system only **suggests** problems. People decide. The cleaner is at the property and can check the drawer. The host owns the final call, because a confirmed loss becomes the evidence record and a false accusation against a cleaner or guest is the worst outcome.

Action on a finding	Cleaner	Host
Run an audit, upload videos	Yes	Yes
Add or edit a note (for example 'it was in the drawer')	Yes	Yes
Mark **found** (the item turned up)	Yes	Yes
Undo their own **found**	Yes	Yes
Confirm loss	No (HTTP 403)	Yes
Dismiss (false alarm)	No (HTTP 403)	Yes
Reopen a finding	Only their own found	Yes

- **Typical flow:** the audit finds a missing remote. The cleaner looks, finds it in a drawer, taps **It was found** and adds a note. The host sees 'Cleaner says it was found, host to review' and taps **Dismiss** or **Confirm loss**.
- **Cleaner cannot find it:** the cleaner leaves it open, the host decides. The host can also confirm directly without the cleaner.

- **No login in v1.** The role is sent with every update (`role: 'cleaner' | 'host'`). The frontend takes it from the link: `/audit/{id}?role=host` for the host (default links from the property page), `/audit/{id}?role=cleaner` for the cleaner (the default when no role is given). The host has a **Copy link for the cleaner** button.
- **This is a stand-in, not security.** Anyone who edits the link can act as host. Before real users, add login, take the role from the signed-in user, and ignore any role sent by the client.
- The server records who resolved each finding (`resolved_by`) and when (`resolved_at`) so the history is clear.

A6b. Inspections, cases and readiness: the rules

These rules are enforced by the server and shown by the screens. Both of you need to know them, so they are here, once.

Action	Cleaner	Host
Start an inspection, set item results, tick cleaning steps, add photos	Yes	Yes
Complete an inspection when all checks are done	Yes	Yes
Complete with an **exception** (needs a written reason)	No (403)	Yes
Add a note, search entry or photo to a case	Yes	Yes
Resolve a case as **found** or **restored_to_place**	Yes	Yes
Resolve as **repaired**, **replaced** or **recorded_missing**	No (403)	Yes
Assign a case, mark it **blocks readiness**	No (403)	Yes

- **Results:** ok, not_seen, misplaced, missing, damaged (and low_stock for consumables, set by the server when the count is under the minimum).
- **Required checks (HTTP 422 if broken):** a high-value item needs a photo before OK. A counted quantity below the expected one cannot be OK. Consumables need a counted quantity and cannot be marked missing.
- **Cases open by themselves** when a result is not_seen, misplaced, missing or damaged. You cannot flip such an item to OK while its case is open (HTTP 409): resolve the case first so the record stays complete.
- **Completing:** blocked (HTTP 409) while checks are pending, with a list of what is left. The host may complete with an exception note. The inspection is then `completed_with_exception`.
- **Readiness:** `ready` = everything done, no open blocking case. `ready_with_exceptions` = completed by exception, no essential gaps. `not_ready` = the inspection is still in progress, or a case marked blocks_readiness is open. An exception never turns a blocking gap into ready.
- **Recorded missing:** a case resolved as recorded_missing sets the item's condition to missing. The next inspection skips it, and the readiness report lists it under known gaps, so nobody is asked to find it again.
- **Not seen is not missing.** Only a person resolves a case as recorded_missing, and only the host.
- **AI hints:** `POST /inspections/{id}/ai-suggestions` copies findings from an AI video check onto the checklist items as `ai_suggestion`. It never changes a result.
- **Wording:** cases are factual ('not found at the usual place'), never accusing. The trail records who did what and when, and entries cannot be edited or deleted.

A7. Running the project

One command starts everything (database, backend, frontend):

```
docker compose up --build
```

What	URL
App	http://localhost:3000
Live API docs (always matches the code)	http://localhost:8000/docs
Health check	http://localhost:8000/api/health

- It works with no `.env` file. To use real models later: `cp .env.example .env`, set `MOCK_MODELS=false`, add keys, run again.
- On the Properties page click **Load demo data (dev)**. You get a ready property with three rooms, nested places, items of every kind and house rules.
- **Upgrading from v1?** The new version adds database columns and tables, and the app does not migrate old data. Run `docker compose down -v` once, then `docker compose up --build`.
- Code changes reload by themselves (backend `--reload`, frontend `next dev`). You only rebuild when dependencies change.
- **Real phone testing:** put the phone on the same Wi-Fi. In `.env` set `NEXT_PUBLIC_API_URL` and `PUBLIC_BASE_URL` to `http://<computer-ip>:8000` and `FRONTEND_ORIGIN` to `http://<computer-ip>:3000`. Restart. Open `http://<computer-ip>:3000` on the phone.
- Reset the database: `docker compose down -v`. Backend logs: `docker compose logs -f backend`.

Part B. The shared API contract (identical in both guides)

Source of truth in code: `backend/app/schemas.py`. **Frontend mirror:** `frontend/lib/types.ts`. **Live version:** http://localhost:8000/docs. A change to any endpoint, field or enum is ONE pull request touching all three of schemas.py, types.ts and docs/API_CONTRACT.md, approved by both people.

B1. Conventions

Topic	Rule
Base URLs	Backend http://localhost:8000. The browser calls it directly using NEXT_PUBLIC_API_URL. Interactive docs at http://localhost:8000/docs are the live version of this contract.
IDs and times	All ids are strings (UUIDs). Timestamps are ISO 8601 strings.
Media URLs	Every *_url field is an absolute URL, ready for an img src. The frontend never builds media URLs.
Boxes	Normalized 0..1 {x, y, w, h}. Multiply by the displayed image size to draw.
Uploads	multipart/form-data. Field is 'file' for one file and 'files' for several. Images max 15 MB, videos max 150 MB.
Errors	Non-2xx responses carry JSON {"detail": "human readable message"}. 400 bad input, 404 not found, 409 audit already running or action not allowed in the current state (inspection completed, case resolved, checks pending), 413 file too large, 422 unreadable video.
Long work	Audits run in the background. POST /run returns 202 at once. Poll GET /audits/{id} every 2 seconds until status is done or failed. The 'stage' text is shown to the user.
Auth	None in v1 (hackathon). Guest access is by the unguessable guest_token. Add host auth after the demo.
Item lifecycle	New scan items are 'suggested'. Only 'confirmed' items are used for guest answers, audits and inspections. Guests never see staff_only items.
Two kinds of check	An AI video check (audit) produces suggestions. An inspection is the human checklist and the only thing that produces results, cases and readiness. The two connect only through POST /inspections/{id}/ai-suggestions, which fills hints.
Cleaning vs inventory	Every room in an inspection has two separate statuses: cleaning_status and inventory_status. Cleaned does not mean complete, and complete does not mean ready.
Consumables	Counted by quantity, never marked missing. Below min_quantity the result becomes low_stock. They are excluded from AI video checks.
Not seen is not missing	'not_seen' means nobody could confirm the item. Only a person (or the host on a case) records 'missing'. The AI never does.
Inspection rules (422 if broken)	High-value items need a photo before OK. A counted quantity below expected cannot be OK. Not seen, misplaced, missing and damaged open a case automatically, and the item cannot be flipped to OK while that case is open (409). Completing is blocked (409) while checks are pending or cases are unresolved, unless the host completes with an exception_note. An exception never hides an essential (blocks_readiness) gap.
Cases	A case is a factual record with an append-only event trail. Statuses: open, investigating, resolved. Cleaners may log searches and notes and resolve only as found or restored_to_place. Only the host may assign, mark blocks_readiness, or resolve as repaired, replaced or recorded_missing (403 otherwise). An item recorded missing is skipped in the next inspection and listed under known_gaps.
Readiness	ready = no blockers and no exception. ready_with_exceptions = host exception, no essential gaps. not_ready = any essential gap, open blocking case, or unfinished room.
Roles on bodies	Every write that has a rule carries role ('cleaner'|'host') and actor (a name). The name is stored for the audit trail. Both are stand-ins until real login exists.

Topic	Rule
Roles (v1, no login)	The client sends role 'cleaner' or 'host' with every finding update. The frontend picks it from the link: /audit/{id}?role=host for the host, ?role=cleaner (the default) for the cleaner. This is a stand-in for real login; it is not secure. Add auth before real users.
Who may resolve a finding	Cleaner: may set 'found' (item turned up) and add a note; may 'open' again only to undo their own 'found'. Cannot confirm or dismiss. Host: may set any resolution, and only the host can confirm a loss. Not allowed -> HTTP 403. Missing role -> HTTP 422. (Applies to AI video check findings. Inspections and cases have their own rules above.)
Finding rules	'unverified' means the camera did not cover the spot, never an accusation. 'missing' is only produced at confidence >= 0.7. 'misplaced' means the item WAS seen but not in its usual place: either elsewhere in its room, or in another room (the observed text says where). 'count_mismatch' means fewer units seen than the saved quantity.

B2. Endpoints

Method	Path	Request	Response	Used in
GET	`/api/health`	-	Health	dev banner
POST	`/api/dev/seed`	-	SeedResult	Properties page button
GET	`/api/properties`	-	PropertyOut[]	Properties list
POST	`/api/properties`	JSON PropertyCreate {name, address?, property_type?, capacity?, details?, rules?}	PropertyOut	Properties list
PATCH	`/api/properties/{id}`	JSON PropertyPatch	PropertyOut	Property page
GET	`/api/properties/{id}`	-	PropertyDetail	Property page
POST	`/api/properties/{id}/rooms`	JSON {name, room_type?, parent_id?}	RoomOut	Property page, Room page
GET	`/api/rooms/{room_id}`	-	RoomOut	Room page
PATCH	`/api/rooms/{room_id}`	JSON {name?, room_type?}	RoomOut	Room page
GET	`/api/rooms/{room_id}/locations`	-	RoomOut[]	Room page
POST	`/api/rooms/{room_id}/photo`	multipart: file (image)	RoomOut	Room page
GET	`/api/rooms/{room_id}/sheet`	-	HTML page	Room page (opens in new tab)
GET	`/api/properties/{id}/rooms`	-	RoomOut[]	Property page
POST	`/api/rooms/{room_id}/baseline/photo`	multipart: file (image), priority (high|normal)	BaselineResult	Room page
POST	`/api/rooms/{room_id}/baseline/video`	multipart: file (video)	BaselineResult	Room page
GET	`/api/rooms/{room_id}/items`	query: status (optional)	ItemOut[]	Room page
PATCH	`/api/items/{id}`	JSON ItemPatch	ItemOut	Room page

Method	Path	Request	Response	Used in
DELETE	`/api/items/{id}`	-	204 no body	Room page
POST	`/api/rooms/{room_id}/items`	JSON ItemCreate	ItemOut	Room page
POST	`/api/items/{id}/condition`	multipart: condition, actor, role, note?, file? (photo)	ItemOut	Room page
GET	`/api/items/{id}/history`	-	ConditionEventOut[]	Room page
POST	`/api/rooms/{room_id}/items/confirm-all`	-	{confirmed: n}	Room page
GET	`/api/guest/{token}`	-	GuestProperty	Guest page
POST	`/api/guest/{token}/ask`	multipart: file (image)	GuestAnswer	Guest page
POST	`/api/properties/{id}/audits`	-	AuditCreated	Property page
POST	`/api/audits/{id}/rooms/{room_id}/video`	multipart: file (video)	UploadAck	Audit page
POST	`/api/audits/{id}/rooms/{room_id}/frames`	multipart: files (images, repeated)	UploadAck	Audit page (optional)
POST	`/api/audits/{id}/run`	-	RunAck (HTTP 202)	Audit page
GET	`/api/audits/{id}`	-	AuditDetail	Audit page (poll every 2 s)
PATCH	`/api/findings/{id}`	JSON {role (required), resolution?, note?}	FindingOut	Audit page
POST	`/api/properties/{id}/inspections`	JSON {kind, inspector, reservation_ref?}	InspectionOut	Property page
GET	`/api/properties/{id}/inspections`	-	InspectionOut[]	Property page
GET	`/api/inspections/{id}`	-	InspectionDetail	Inspection page
PATCH	`/api/inspection-items/{id}`	JSON InspectionItemPatch	InspectionItemOut	Inspection page
POST	`/api/inspection-items/{id}/photo`	multipart: file, actor, role	InspectionItemOut	Inspection page
PATCH	`/api/cleaning-checks/{id}`	JSON {actor, done}	CleaningCheckOut	Inspection page
POST	`/api/inspections/{id}/ai-suggestions`	JSON {audit_id}	{applied: n}	Inspection page
POST	`/api/inspections/{id}/complete`	JSON {role, actor, exception_note?}	InspectionOut	Inspection page
GET	`/api/inspections/{id}/readiness`	-	ReadinessReport	Readiness page
GET	`/api/properties/{id}/discrepancies`	query: status (optional)	DiscrepancyOut[]	Cases page
GET	`/api/discrepancies/{id}`	-	DiscrepancyDetail	Case page
PATCH	`/api/discrepancies/{id}`	JSON {role, actor, status?, assigned_to?, blocks_readiness?}	DiscrepancyOut	Case page

Method	Path	Request	Response	Used in
POST	`/api/discrepancies/{id}/events`	multipart: role, actor, kind, text?, file?	DiscrepancyEventOut	Case page
POST	`/api/discrepancies/{id}/resolve`	JSON {role, actor, outcome, note?}	DiscrepancyOut	Case page
GET	`/media/{path}`	-	image bytes	all pages (img src)

B3. Data shapes

Name	Shape
`BBox`	{ x, y, w, h } numbers 0..1, relative to the image width and height (top-left origin)
`RoomOut`	{ id, property_id, parent_id|null, name, room_type, path ('Living room > Closet'), reference_photo_url|null, confirmed_items, suggested_items }. A place inside a room has parent_id set
`PropertyOut`	{ id, name, guest_token, created_at, address|null, property_type|null, capacity|null, details|null, rules|null }
`PropertyDetail`	PropertyOut + { rooms: RoomOut[] (top-level only), latest_audit_id|null, latest_inspection_id|null, open_discrepancies }
`ItemOut`	{ id, room_id, label, category|null, quantity, priority, bbox|null, crop_url|null, source_image_url|null, location_hint|null, notes|null, status 'suggested'|'confirmed'|'retired', location_id|null, location_path|null, kind 'fixed'|'movable'|'shared'|'consumable', condition 'new'|'good'|'worn'|'damaged'|'missing'|'under_repair', availability 'guest'|'staff_only', high_value, min_quantity|null, alternate_locations: string[] }
`ItemCreate`	{ label, quantity? (1..999), kind?, category?, location_id?, location_hint?, notes?, availability?, high_value?, min_quantity?, alternate_locations? }
`ItemPatch`	any of { label, quantity (1..999), notes, location_hint, status, kind, availability, high_value, min_quantity, alternate_locations, location_id (null = anywhere in the room) }
`ConditionEventOut`	{ id, item_id, condition, previous|null, note|null, photo_url|null, actor, role|null, created_at }
`BaselineResult`	{ capture_id, items: ItemOut[] } (new items arrive with status 'suggested')
`GuestProperty`	{ property_name, item_labels: string[], house_rules|null } (staff_only items are never listed)
`GuestAnswer`	{ matched, message, item_label|null, guide: GuestGuide|null }
`GuestGuide`	{ title, steps: string[], warnings: string[], seen, unsure: string[] }
`AuditCreated`	{ id, property_id, status }
`UploadAck`	{ audit_id, room_id, frames }
`RunAck`	{ audit_id, status }
`FindingOut`	{ id, audit_id, item_id|null, room_id|null, item_label, type 'missing'|'misplaced'|'count_mismatch'|'extra'|'unverified', expected|null, observed|null, confidence|null, evidence_frame_url|null, reference_crop_url|null, resolution 'open'|'found'|'confirmed'|'dismissed', resolved_by 'cleaner'|'host'|null, resolved_at|null, note|null }
`AuditDetail`	{ id, property_id, status 'draft'|'running'|'done'|'failed', stage|null, summary|null, rooms_uploaded: room_id[], findings: FindingOut[], created_at }
`SeedResult`	{ property_id, guest_token, room_ids[] }
`Health`	{ status, mock_models }

Name	Shape
`InspectionOut`	{ id, property_id, kind 'after_checkout'|'after_cleaning'|'before_checkin', inspector, reservation_ref|null, status 'in_progress'|'completed'|'completed_with_exception', exception_note|null, exception_by|null, created_at, completed_at|null }
`AiSuggestion`	{ type (a finding type), observed|null, confidence|null, evidence_frame_url|null, audit_id } (a hint, never a result)
`InspectionItemOut`	{ id, inspection_id, room_id, item_id|null, label, kind, high_value, expected_quantity, min_quantity|null, result 'pending'|'ok'|'not_seen'|'misplaced'|'missing'|'damaged'|'low_stock', observed_quantity|null, note|null, photo_url|null, checked_by|null, checked_at|null, reference_crop_url|null, location_path|null, alternate_locations[], ai_suggestion|null, discrepancy_id|null }
`CleaningCheckOut`	{ id, room_id, label, done, done_by|null, done_at|null }
`RoomInspection`	{ room_id, room_name, room_type, cleaning_status, inventory_status ('pending'|'in_progress'|'done'|'not_applicable'), items: InspectionItemOut[], cleaning: CleaningCheckOut[] }
`InspectionDetail`	InspectionOut + { rooms: RoomInspection[], blockers: string[], can_complete }
`InspectionItemPatch`	{ role, actor, result?, observed_quantity?, note? }
`ReadinessReport`	{ inspection_id, property_name, kind, inspector, reservation_ref|null, status 'ready'|'ready_with_exceptions'|'not_ready', cleaning_status, inventory_status, summary, rooms: [{room_name, cleaning_status, inventory_status, issues[]}], blocking[], open_cases[], known_gaps[], low_stock[], exception_note|null, exception_by|null, generated_at }
`DiscrepancyOut`	{ id, property_id, inspection_id|null, item_id|null, room_id|null, room_name|null, item_label, type 'missing'|'misplaced'|'damaged'|'not_seen', status 'open'|'investigating'|'resolved', assigned_to|null, blocks_readiness, outcome 'found'|'restored_to_place'|'repaired'|'replaced'|'recorded_missing'|null, resolution_note|null, resolved_by|null, resolved_at|null, created_at }
`DiscrepancyEventOut`	{ id, kind 'detected'|'note'|'search'|'evidence'|'assigned'|'status'|'resolved', text|null, photo_url|null, actor, role|null, created_at }
`DiscrepancyDetail`	DiscrepancyOut + { events: DiscrepancyEventOut[], reference_crop_url|null, expected_location|null, ai_suggestion|null }

B4. Mock mode: what the frontend can rely on

- MOCK_MODELS=true is the default. No keys needed. Answers are fake but consistent, so every screen can be built.
- POST /api/dev/seed creates 'Demo loft (seeded)' with a Living room, a Kitchen and a Bathroom, nested places (Living room > Glass cabinet > Top shelf, Kitchen > Upper cupboard > Left shelf, Under-sink cupboard), a mix of item kinds (movable, fixed, consumable), high-value items, items with an alternate place, and one suggested item to review. It also stores house rules.
- Seeded AI video check behaviours (5 findings): 'Black Roku remote' is missing (searched in the other room too, still missing), 'Samsung TV remote' is misplaced (seen on the sofa, usually on the coffee table), 'Bottle opener' is misplaced (kitchen item found in the Living room), 'Wine glass set' is count_mismatch (3 of 4), 'Smart speaker' is unverified. The rest pass. The TV remote's alternate place is 'TV console', not 'sofa', so it stays misplaced.

- Any item whose notes contain [mock:missing], [mock:unverified], [mock:count=N], [mock:moved=some place] or [mock:otherroom] triggers that outcome in an AI video check, so the frontend can create its own test cases by editing notes. ([mock:otherroom] needs a second room with an uploaded video in the same audit.)
- Inspections need no model: they are human checklists, so the whole inspection, case and readiness flow works the same with or without mock mode.
- Any video works in mock mode, as long as it is a real video file (ffmpeg extracts frames from it).

B5. Example calls (copy and run)

```
# create a property and a room
curl -s -X POST localhost:8000/api/properties -H 'Content-Type: application/json' -d '{"name":"Beach flat"}'
curl -s -X POST localhost:8000/api/properties/<PID>/rooms -H 'Content-Type: application/json' -d '{"name":"Kitchen

# baseline photo -> suggested items
curl -s -F file=@kitchen.jpg localhost:8000/api/rooms/<ROOM_ID>/baseline/photo

# confirm everything the model suggested
curl -s -X POST localhost:8000/api/rooms/<ROOM_ID>/items/confirm-all

# audit: create, upload a video for a room, run, poll
curl -s -X POST localhost:8000/api/properties/<PID>/audits
curl -s -F file=@walk.mp4 localhost:8000/api/audits/<AID>/rooms/<ROOM_ID>/video
curl -s -X POST localhost:8000/api/audits/<AID>/run
curl -s localhost:8000/api/audits/<AID>

# inspection: start, set one result, complete, read the report
curl -s -X POST localhost:8000/api/properties/<PID>/inspections -H 'Content-Type: application/json' -d '{"kind":"a
curl -s localhost:8000/api/inspections/<IID>
curl -s -X PATCH localhost:8000/api/inspection-items/<IIID> -H 'Content-Type: application/json' -d '{"role":"clean
curl -s -X POST localhost:8000/api/inspections/<IID>/complete -H 'Content-Type: application/json' -d '{"role":"hos
curl -s localhost:8000/api/inspections/<IID>/readiness
```

Part C. How a video confirms the photos you took earlier

Remember: everything in this part produces **suggestions**. A finding from the AI video check never becomes a result, a case or a readiness decision until a person acts on it (A6 and A6b). **The question:** the host photographs certain items at scan time. Later a video is recorded. Can the system tell, from the video, whether each photographed item is still there? **Yes. This is exactly what the audit does**, in six steps.

Step	What happens	Where
1. Save references	At scan time every confirmed item keeps its **reference crop** (close photo of the item), the **source photo** (wide shot showing where it sits), its label, quantity, location hint and host notes.	Baseline pipeline, tables `items`
2. Keyframes	The room video is cut into still frames, 2 per second, capped at 40 per room.	`services/frames.py` (ffmpeg)
3. Shortlist frames	For each item, pick the 4 frames most likely to show it, by labeling objects in a sample of frames and ranking frames whose labels resemble the item (the match is by what the item looks like, not where it is). If nothing matches, use 4 evenly spread frames.	`pipeline/matching.py`, `pipeline/verdicts.py`
4. Judge	Send the reference crop, the wide source photo and the frames to the vision model. It answers **present**, **not_visible** or **missing**, plus a count, the best evidence frame, a confidence, **where it saw the item and whether that is its usual spot**, and one sentence of reasoning.	`client.judge(...)` in `services/models`
5. Second look and other rooms	Not clearly present? Look again with 8 frames; `missing` only survives if both passes agree. Still not present? Search the frames of the other rooms in the same audit (up to 3 rooms).	`check_item()` in `pipeline/verdicts.py`
6. Turn the answer into a finding	Seen elsewhere (this room or another) -> `misplaced`. Missing at confidence 0.7 or more -> `missing`. Fewer seen than the saved quantity -> `count_mismatch`. Not visible or low confidence -> `unverified`. Present in its place -> no finding. A person resolves every finding (A6).	`check_item()` in `pipeline/verdicts.py`

What if an item was moved?

The system recognises an item by **how it looks** (the reference crop), not by where it sits, so a moved item can still be found if the video shows it. What happens depends on where it ended up:

Where the item is now	Result	Example text on the finding
Elsewhere in the same room, filmed	`misplaced`	Seen on the sofa, left cushion. Usually: On the coffee table.
In another room that was filmed in the same audit	`misplaced`	Found in Living room, on a surface. It belongs in Kitchen.
Anywhere that was not filmed (drawer, bag, off-camera)	`missing` or `unverified`	Not seen in the walkthrough. (Unverified if the area was not covered.)
Still in its usual spot	no finding	-

- **To catch a move into another room, film that room in the same audit.** The cross-room search only uses frames uploaded to the current audit, and checks at most 3 other rooms.
- **Alternate places:** if the host lists another acceptable place for an item (the remote on the TV console), seeing it there is **not** reported as misplaced.

- **Consumables are excluded** from the AI video check. They are counted by a person in an inspection.
- A `misplaced` finding is not a loss. The cleaner can tap **I put it back**, the host can confirm that it moved or dismiss it.
- Moved items are the hardest case for the model: it must be sure it is the same object, not a similar one. Confidence below 0.6 is ignored, which means a real move can occasionally go unreported. The test set (backend guide D7) measures this.

What the user sees

Each finding card shows the **baseline photo** next to the **evidence frame** from the video, so the person can judge in one second. Model confidence is shown as a secondary detail, never as the proof.

What this can and cannot do (be honest in the demo)

Works well	Weak spots, and how the design handles them
Distinct, reasonably large items: TV remote, coffee machine, lamp, router, a set of glasses in plain view.	**Identical-looking items** (two white remotes): the model may call the wrong one present. Mitigation: location hints, host notes, human confirmation.
Items in the same spot as in the scan.	**Hidden items** (inside a drawer, behind a cushion): not visible, so the answer must be `unverified`, never `missing`. The cleaner opens the drawer on camera.
Well-lit, slow, steady video that pauses on each spot.	**Blur, glare, dark rooms, fast panning.** Mitigation: capture tips in the UI, `unverified` when unsure, re-film prompt.
Counting 2 to 6 clearly separated objects.	**Large counts or stacked items** (towels): counts are unreliable. Host sets the quantity; flag only when the shortfall is clear.

> **Accuracy is measured, not assumed.** The mock mode only proves the plumbing. Real accuracy depends on the real models, which are not tested yet. The backend dev must run the test set in section D7 of the backend guide and report real numbers before the demo.

Capture rules that make matching work (frontend shows these as tips, host and cleaner follow them)

- **Reference photo:** one close photo per important item, item filling at least a third of the frame, plus one wide photo showing where it normally sits.
- **Open drawers and cabinets fully** and hold still for one second, in the scan and in the audit.
- **Audit video:** walk the same route each time, 20 to 30 seconds per room, pause one second at every shelf, table and counter, good light, no flash.
- **Re-scan** after the host rearranges the room or buys new items.

Part D. Frontend-only guide: you build the app people touch

D1. Your mission

Build a fast, phone-first web app for three kinds of people: the **host** (records the home, reviews problems, decides readiness), the **guest** (asks about an appliance) and the **cleaner** (walks the inspection checklist and optionally films a room for AI hints). You own everything in `frontend/`. You never need model keys, a database, or the backend code: Part B and mock mode give you everything.

In scope	Out of scope (do not build)
All screens in D4, capture flows, upload states, polling, inspections, cases, readiness report, guest page, QR code, PWA touches.	Login and accounts, bookings and calendar, notifications, stock ledger, maintenance jobs, dashboards, exports, offline mode, native apps, anything the contract does not list (see A5).

D2. Stack and folder map

Tool	Version	Why
Next.js (App Router) + React + TypeScript	14.2 / 18 / 5	Pages, routing, one language
Tailwind CSS	3.4	Fast mobile styling, no CSS files to manage
Plain fetch in `lib/api.ts`	-	No extra client library, one place to change

```
frontend/
  app/
    layout.tsx                        page shell, fonts, viewport
    page.tsx                          redirects to /properties
    properties/page.tsx               list + create + "Load demo data"
    properties/[id]/page.tsx          details, rooms, start inspection, cases link, guest link
    properties/[id]/rooms/[roomId]/page.tsx   photo, places, scan + review, item details, condition
    properties/[id]/cases/page.tsx    list of cases
    cases/[caseId]/page.tsx           one case: trail, evidence, assign, resolve
    inspection/[id]/page.tsx          room-by-room checklist, complete / exception
    inspection/[id]/readiness/page.tsx  printable readiness report
    audit/[auditId]/page.tsx          AI video check: record per room, run, suggestions
    g/[token]/page.tsx                guest: house rules, snap a photo, read steps
  components/
    BBoxOverlay.tsx                   boxes over an image (0..1 coordinates)
    FilePicker.tsx                    opens phone camera for photo or video
    Shell.tsx                         page frame, back link, error note, RoleBar, Pill
  lib/
    api.ts                            EVERY backend call lives here
    types.ts                          mirror of the contract (shared file)
    usePolling.ts                     poll until a condition is met
    useRole.ts                        ?role=host|cleaner, remembered name (actor)
```

Pages that read `?role=` must be wrapped in `<Suspense>` (Next.js requirement for `useSearchParams`). The inspection, case and audit pages already are. Copy that pattern.

D3. Working without waiting for anyone

- Run `docker compose up --build`, open http://localhost:3000, press **Load demo data (dev)**. You now have a full property to design against.
- Mock mode returns believable data. To get each audit outcome on demand, edit an item's notes (the Notes box on the room page) and add `[mock:missing]`, `[mock:unverified]` or `[mock:count=2]`.

- Upload any real video (phone clip, 5+ seconds) to test audit uploads. Mock mode accepts anything that is a real video file.
- To test a slow backend, add a temporary delay in your own code (for example `await new Promise(r => setTimeout(r, 3000))` in `api.ts` during development) and remove it before merging.
- Open http://localhost:8000/docs to try any endpoint by hand and see the exact response.
- If you need a field or endpoint that is missing: open a `contract/` PR (edit `types.ts` and `API_CONTRACT.md`) and tag the backend dev. Do not fake it permanently in the UI.

D4. Screens: what each one must do

Screen 1. Properties list (/properties)

API calls	`GET /properties`, `POST /properties`, `POST /dev/seed`
Must show	List of properties (type, capacity, address), create form (name, address, type, guests), demo-data button (dev only), empty state, error state.
Done when	Create a property, see it listed, open it. Works at 375 px wide.

Screen 2. Property (/properties/[id])

API calls	`GET/PATCH /properties/{id}`, `POST /properties/{id}/rooms`, `GET/POST /properties/{id}/inspections`, `POST /properties/{id}/audits`
Must show	Details and house rules (view and edit), rooms with type and counts, add room with a type, **Start an inspection** (three kinds, reservation reference, the person's name), recent inspections with status, link to Cases with the open count, **Start AI video check (suggestions only)**, guest link with copy button and QR code.
Done when	A host can set the rules, add Kitchen and Living room, start an after-checkout inspection that opens the checklist, and copy the guest link.
Stretch	QR code: run `docker compose exec frontend npm i qrcode.react`, render the guest link as a QR the host can print.

Screen 3. Room: scan and review (/properties/[id]/rooms/[roomId])

API calls	`GET /rooms/{id}`, `GET /rooms/{id}/locations`, `POST /rooms/{id}/photo`, `POST baseline/photo`, `POST baseline/video`, `GET/POST items`, `PATCH/DELETE items/{id}`, `POST items/confirm-all`, `POST items/{id}/condition`, `GET items/{id}/history`, `GET /rooms/{id}/sheet` (opens in a new tab)
Must show	Two capture buttons (item photo, 10-second sweep). After upload: the photo with a labeled box on every suggested item (`BBoxOverlay`), then one card per suggested item with crop, editable name, quantity stepper, **host note** box, Confirm and Delete. A **Confirm all** button. A second list of confirmed items.
States	Uploading (button disabled, visible progress text), model working (can take 5 to 30 s, show 'Finding items...'), no items found (friendly message, suggest closer photo), error with Retry.
Done when	Take a photo, see boxes, rename one item, set quantity 4, add a note, confirm all, and the counts on Screen 2 update.
Also on this screen	Room reference photo (upload and replace), **Print room sheet** link, places inside the room (add; shown with their full path), add an item by name, and per item a **Details** panel: kind, who can see it, usual place, other acceptable places, high value, minimum quantity (consumables), condition buttons with an optional note and a **Show history** list.

Why notes matter	The host note ('Fill water at the back right, push the silver lever up') is the best source for guest instructions. Make the box easy to find.

Screen 4. AI video check (/audit/[auditId])

> This is the **suggestions-only** check. Its title says so. It never changes an inspection by itself.

One screen with three phases, driven by `status` from `GET /audits/{id}`:

status	Show	Calls
draft (or failed)	Capture tips. One card per room with **Record video** and an 'Uploaded' tick. **Run audit** disabled until at least one room is uploaded. If failed, show `stage` as the error.	`POST audits/{id}/rooms/{room}/video`, `POST audits/{id}/run`
running	Progress card with the `stage` text (it changes: 'Finding items in Kitchen', 'Checking Kitchen: Nespresso (2/5)'). Poll every 2 seconds. Keep the screen awake feel: no spinner-only states.	`GET audits/{id}` every 2 s
done	Summary line, then finding cards sorted: missing, count mismatch, misplaced, unverified. Each card: type badge, item name, room, expected vs observed, **baseline photo and evidence frame side by side**, a note box, a status line, and buttons that depend on the role (below). Unverified cards say 'Please re-film this area'.	`PATCH findings/{id}` (always send `role`)

Role views (see A6). The same screen is used by two people. The role comes from the URL: `?role=host` or `?role=cleaner` (default). Show 'Viewing as host/cleaner' at the top and send the role with every `PATCH /findings/{id}`.

Finding state	Cleaner sees	Host sees
open	Note box and **It was found** (for `misplaced`: **I put it back**)	Note box, **Confirm loss** (for `misplaced`: **Confirm it moved**), **Dismiss**
found (by cleaner)	Status 'Cleaner says it was found. Host to review.' and **Undo**	Same status, **Confirm loss**, **Dismiss**, **Reopen**
confirmed	Status 'Confirmed as a loss by the host.' No buttons.	Status, **Dismiss**, **Reopen**
dismissed	Status 'Dismissed by the host.' No buttons.	Status, **Confirm loss**, **Reopen**

- The server answers HTTP 403 if a cleaner tries to confirm or dismiss. Show its `detail` text; do not hide it. The buttons above already prevent this, the 403 is the safety net.
- The host's property page links to `/audit/{id}?role=host`. The host's audit screen has **Copy link for the cleaner** (`?role=cleaner`) so the cleaner opens the right view on their phone.
- `misplaced` cards say what was seen and where ('Found in Living room, on a surface. It belongs in Kitchen.') in plain words. Show the evidence frame: it is the proof of where the item is now.
- Tell the cleaner to film **every room** in the audit: moved items are found in other rooms' frames. Show this tip on the capture phase.
- Closed findings (confirmed, dismissed) sort below open ones and are dimmed. Keep them visible: they are the record.
- Notes save on blur and travel with the next update. A cleaner's note should stay visible to the host.

- A video can be 30 to 100 MB. Show 'Uploading...' on that room's button and block double taps. Do not run the audit while an upload is in progress.
- **Never** word a finding as an accusation. Use 'not found in the video', not 'stolen'.
- Offer a **Copy summary** button on the report (plain text the host can paste into a message).

Screen 5. Inspection (/inspection/[id]?role=...)

API calls	`GET /inspections/{id}`, `PATCH /inspection-items/{id}`, `POST /inspection-items/{id}/photo`, `PATCH /cleaning-checks/{id}`, `POST /inspections/{id}/ai-suggestions`, `POST /inspections/{id}/complete`
Must show	Role bar with the person's name. Per room: the **cleaning status** and **inventory status** as two separate pills, the cleaning checkboxes, then one card per item with reference crop, expected quantity, usual place and alternate places, badges (photo required, consumable, current result), buttons **OK, Not seen, Misplaced, Missing, Damaged** (consumables: a counted-quantity box and **Save count**), a photo button, and a link to the case when one exists.
Hints	If the item has `ai_suggestion`, show it in a blue box labelled 'AI hint (not a result)'. A button **Use hints from the latest AI video check** (shown when `latest_audit_id` exists) calls `ai-suggestions`.
Finish	A **Still to do** list from `blockers`. **Complete inspection** is disabled until `can_complete`. In host view, a textarea and **Complete with exception** (needs a reason). In cleaner view, say only the host can approve an exception. After completing, show **Open readiness report**.
Errors	422 and 409 come with a readable `detail` (photo needed, count below expected, resolve the case first). Show it as is.
Done when	Seed a property, start an inspection, mark the high-value item OK without a photo (see the 422), add the photo, mark OK, mark another item Missing (a case appears), complete.

Screen 6. Readiness report (/inspection/[id]/readiness)

API calls	`GET /inspections/{id}/readiness`
Must show	Property, a big status pill (ready, ready with exceptions, not ready), the `summary` sentence, who inspected and when, the two statuses, the exception reason when present, lists for blocking items, open cases, known gaps and low stock, a card per room with its issues, and a **Print** button (`window.print()`).
Rule	'Inspection complete' and 'ready' are different things. Never show a green tick just because the inspection is complete.

Screen 7. Cases (/properties/[id]/cases and /cases/[caseId])

API calls	`GET /properties/{id}/discrepancies`, `GET /discrepancies/{id}`, `PATCH /discrepancies/{id}`, `POST /discrepancies/{id}/events`, `POST /discrepancies/{id}/resolve`
List	Cards with item, status pill, type, assignee, outcome, and a red **blocks readiness** pill. A **Show resolved** toggle. Neutral wording: a case is a record, not a blame list.
Detail	Header with status, expected place, reference crop and any AI hint. The **History** list (kind, who, role, time, text, photo). **Add to the record**: textarea, **Log search**, **Add note**, **Add photo evidence**.
Host only	Assign (name box + button), **Blocks readiness** checkbox. In cleaner view these controls are not shown.
Resolve	Outcome select and a note. The cleaner's select offers only found and restored to place. The host's offers all five. Resolved cases show the outcome and hide the editing controls.

Done when	Open the Missing case from the inspection, log a search, add a photo, resolve as found as cleaner; the inspection can then be completed.

Screen 8. Guest (/g/[token])

API calls	`GET /guest/{token}`, `POST /guest/{token}/ask`
Must show	Property name, the **house rules** when set (short box), one big **Snap a photo** button, the photo preview, then either numbered step cards (title, steps, warnings in a yellow box, 'not sure about...') or a friendly 'could not match' message with the option to try another angle.
Rules	No login, no menus, one screen, works on a slow connection. Guests are on holiday and tired: large text, short sentences. Add a **Not right?** link that opens a mailto or shows the host's contact (stretch).
Done when	A guest with a link can photograph the seeded coffee machine and read the steps in under 15 seconds.

D5. Capture and upload rules

- Use the native camera input (`<input type=file accept=... capture=environment>`). It opens the phone camera on iOS and Android with no permission code. `FilePicker.tsx` already does this. Do not switch to `getUserMedia` unless you finish everything else.
- Send the video file as is to `POST /audits/{id}/rooms/{room}/video`. The backend cuts frames. You do not need ffmpeg in the browser. (`/frames` exists as a fallback only.)
- Limits: images 15 MB, videos 150 MB. Show a clear message on HTTP 413 ('Please record a shorter video'). On 422 say 'We could not read that video, please try again'.
- Phones often record HEVC `.mov`. That is fine, the backend handles it. Do not convert in the browser.
- Always show the file name or a thumbnail after upload so the user trusts that it worked.
- Use `URL.createObjectURL(file)` for instant previews before the server answers.
- Never block the UI on the long model calls. Baseline calls return when done (up to ~30 s): show a friendly waiting state. The audit is the only background job: poll it.

D6. Using the API correctly

- **All backend calls go through** `lib/api.ts`**.** Pages never call `fetch` for the backend directly. Add a new function there when the contract grows.
- **Types come from** `lib/types.ts`**.** If TypeScript and the response disagree, the response is right: tell the backend dev or fix the type in a contract PR.
- **Media URLs are absolute.** Put `item.crop_url` straight into `<img src>`. Never prefix or rebuild them.
- **Errors:** `api.ts` already throws `Error(detail)`. Catch it, show it in an `ErrorNote`, and give a way to retry.
- **Boxes** are 0..1 fractions of the image: position them with percentages (see `BBoxOverlay.tsx`), so they stay right at any screen size.
- **Polling:** use `usePolling` and stop on `done` or `failed`. Do not poll faster than every 2 seconds.
- **Do not store secrets or API keys** in the frontend. There are none. Only `NEXT_PUBLIC_API_URL` is configured.

D7. Design rules

Rule	Detail
Phone first	Design at 375 px width. Check 360 and 414. Desktop is a bonus: centered column, max width 672 px.
Tap targets	At least 48 px high for anything pressed. Space buttons apart. (`btn` classes already do this.)
Every screen has 4 states	Loading, empty, error, success. Review each screen for all four before calling it done.
One accent colour	Teal brand colour for the main action. Red only for 'missing', amber for 'count' and 'unverified'. Do not rely on colour alone: also show the label text.
Real photos everywhere	Show the host's own photos and boxes. They are the demo.
Plain words	Short sentences, no jargon ('segment', 'embedding'). Say 'items we found', 'could not check'.
Accessible	Alt text on images, labels on inputs, readable contrast, works with large system text.
Speed	No big libraries. Use plain `<img>`. Keep each page under ~200 lines; split components when it grows.

D8. Your 10-day plan

Days	Goal	Deliverables	Gate to pass
1-2	Foundation	Run the stack on your machine and on your phone (LAN). Properties and Property screens with details, rules, room types; loading/empty/error states, copy-link button.	Create property, set rules, add 2 rooms, open from phone.
3-4	Rooms and items	Room screen: capture buttons, BBoxOverlay, item cards with the Details panel, places, condition and history, reference photo, print sheet.	Photo -> boxes -> confirm -> set a place and a condition -> sheet prints.
5	Guest guide	Guest page with house rules, step cards, warnings, unmatched state, QR code.	Guest gets steps for the seeded coffee machine.
6-7	Inspection, cases, readiness	Inspection checklist (results, counts, photos, cleaning checks, hints, complete and exception), cases list and detail, readiness report.	Demo script in D9 passes: ready, ready with exceptions and not ready all reachable.
8	AI video check	Audit screen as suggestions: room cards, upload, Run, stages, evidence pair, role buttons.	Seeded check shows all 5 findings.
9-10	Polish and demo	Skeleton loaders, copy edits, PWA manifest + icon, accessibility pass, screenshot of every screen, rehearse demo with backend dev.	Three clean rehearsals on a real phone.

D9. Test script (run before every merge)

- [ ] `docker compose up --build` starts clean from scratch (`docker compose down -v` first once a day, and always after pulling a contract change).
- [ ] Load demo data; open property; open Living room; the suggested lamp appears with a box.
- [ ] Upload a phone photo: items appear, you can edit and confirm them.
- [ ] Open the guest link on a phone; photograph any object; steps or a friendly 'no match' appear.

- [ ] Start an after-checkout inspection from the property page. Mark the high-value item OK: you see the photo requirement. Add a photo, then OK. Mark another item **Missing**: a case appears. Try **Complete**: it is disabled and lists what is left.
- [ ] Open the case as cleaner, log a search, resolve as found. Back on the inspection, finish the other items and complete it. Open the readiness report: it says ready.
- [ ] Repeat, but mark an item missing, switch to `?role=host`, tick **Blocks readiness** on its case, complete with an exception note: the report says **not ready**. Resolve the case as recorded missing: now ready with exceptions, and the next inspection skips that item.
- [ ] As cleaner, there is no Complete with exception button and the outcome list has only two options.
- [ ] Start an audit; upload a real video for BOTH rooms (the Bottle opener is found in the other room, so both rooms are needed); Run; progress text changes (including 'Looking for ... in Living room'); report shows 5 findings from the demo data: missing, count mismatch, unverified, and two misplaced (one seen on the sofa, one found in another room).
- [ ] Open the audit with `?role=cleaner`: tap **It was found** on the missing remote and add a note. Switch to `?role=host`: you see the cleaner's status and note; tap **Confirm loss**; reload, it stays confirmed.
- [ ] As cleaner, there is no Confirm or Dismiss button anywhere. (Developer check: calling the PATCH with `role: cleaner, resolution: confirmed` must return 403.)
- [ ] Turn Wi-Fi off mid-upload: you see a clear error, not a blank screen.
- [ ] Every screen at 375 px wide: no horizontal scrolling, no cut-off buttons.

D10. Demo polish list (days 9 and 10)

- A clean first screen with the product name and one sentence of purpose.
- Show the cleaner-then-host story in the demo: cleaner taps found, host dismisses. It shows the system defers to people.
- Findings sorted by severity and a summary line at the top ('2 issues, 1 could not be verified, 3 OK').
- Baseline photo and evidence frame at the same size, side by side.
- A visible 'Demo mode' badge when `GET /health` says `mock_models: true`, so nobody mistakes fake answers for real ones.
- A screen recording of a perfect run as a backup for the demo.

D11. Troubleshooting

Symptom	Likely cause and fix
Browser console says CORS error	`FRONTEND_ORIGIN` in `.env` does not match the URL in your address bar (localhost vs IP). Fix it and restart the backend.
Images are broken	`PUBLIC_BASE_URL` points to the wrong host for your device. On a phone it must be the computer's IP, not localhost.
Phone cannot open the app	Not on the same Wi-Fi, or firewall blocks ports 3000 and 8000.
`Module not found` after adding a package	Run `docker compose exec frontend npm i <pkg>` then commit `package.json`. If still stuck: `docker compose build --no-cache frontend`.
Page shows stale data	The audit and items change on the server. Call `refresh()` or reload after actions.

Symptom	Likely cause and fix
Upload fails with 413	File over the limit. Ask for a shorter video.
Port already in use	Another app uses 3000, 8000 or 5432. Stop it or change the port mapping in `docker-compose.yml` (ask the backend dev).